FORMATION · CONCEPTS

Comprendre les tests logiciels

Protéger les règles métier de notre application



Exemples : notre API Laravel

LE PROBLÈME

Une modification peut casser une règle

L’application accepte soudain un document sans titre.
Ou Karim peut consulter le document d’Amina.

Comment repérer ce changement avant les utilisateurs ?

Une erreur peut apparaître après une modification qui semblait simple

DÉFINITION

Qu’est-ce qu’un test ?

Un test vérifie un comportement précis.

On prépare une situation, on réalise une action
et on compare le résultat obtenu au résultat attendu.

Exemple : une création sans titre doit être refusée

EXEMPLE

Une situation, une action, un résultat

Situation : Amina est connectée.
Action : elle envoie un document sans titre.
Résultat attendu : l'API refuse la création.

Dans notre API : réponse HTTP 422

MÉTIER

Une règle métier à protéger

Un document doit avoir un titre et une catégorie.

Le test décrit ce qui doit arriver
lorsque ces informations manquent.

La règle répond à un besoin de l’application

EXEMPLE

Données acceptées et refusées

Titre et catégorie valides : création acceptée.
Titre absent : création refusée.
Catégorie absente : création refusée.

Dans notre API : 201 pour la création, 422 pour la validation

Chaque personne voit ses documents

Amina possède un document.
Karim se connecte et demande ce document.

Résultat attendu : l’accès est refusé.



La confidentialité est une règle essentielle

Détecter une régression

Une fonction marchait hier.
Une modification la casse aujourd'hui.

Un test relancé peut signaler cette régression.



Les tests protègent les comportements qu'ils vérifient

Test manuel et test automatisé

Manuel : une personne réalise les étapes
et observe le résultat, par exemple avec Postman.

Automatisé : un programme exécute
les étapes et vérifie le résultat.

Les deux approches peuvent se compléter

À différentes échelles

Unitaire : vérifier une petite règle isolée.
Fonctionnel : vérifier un comportement complet,
par exemple une requête vers une route de l'API.

Intégration : vérifier la collaboration de composants.

Le choix dépend du comportement à vérifier

STRUCTURE

Les trois étapes d’un test

Préparer : créer le contexte et les données utiles.
Agir : déclencher une action précise.
Vérifier : comparer le résultat à l’attendu.

Un test clair explique la règle qu’il protège

Tester avec des données adaptées

Utiliser des comptes et documents fictifs.
Prévoir un contexte contrôlé pour chaque essai.
Vérifier que les essais ne polluent pas les données réelles.

Un test doit pouvoir être rejoué de manière fiable

Commencer par les règles attendues

Que doit pouvoir faire l'utilisateur ?
Que doit refuser l'application ?
Quels cas sont les plus importants ?



Écrire des exemples rend les attentes plus claires

Écrire le test avant le code : TDD

1. Écrire un test pour un comportement.
2. Vérifier qu'il échoue pour la raison attendue.
3. Développer jusqu'à ce qu'il réussisse.
4. Améliorer le code en conservant les tests verts.

Le TDD est une méthode possible, pas une obligation

Le titre est obligatoire

Rouge : le test échoue, le refus manque encore.
Vert : l'API rejette maintenant le titre absent.
Amélioration : le code est clarifié,
puis les tests sont relancés.

Le scénario protège ensuite la règle contre une régression

Et si l’application existe déjà ?

Identifier une règle importante.
Ajouter un test sur le comportement actuel attendu.
Puis modifier le code et relancer les tests.

On peut ajouter des tests progressivement

Ce qu’un test ne garantit pas

Un test couvre seulement les cas prévus.
Un test mal conçu peut donner une fausse confiance.
Une règle métier mal comprise peut être testée… et fausse.

Relire les scénarios et les compléter reste nécessaire

Les tests dans notre projet

Laravel peut exécuter des tests automatisés.
Les tests fonctionnels peuvent appeler notre API.
Ils vérifient notamment les réponses et les données.

Aujourd'hui : comprendre les principes avant la pratique

Et les performances avec k6 ?

Un test fonctionnel vérifie le comportement.
Un test de charge observe l'application
avec plusieurs demandes sur une durée donnée.

Les deux répondent à des questions différentes

À RETENIR

Les règles guident les tests

Définir ce que l'application doit faire.
Tester les cas importants et les refus attendus.
Relancer les vérifications après les changements.

Question : quelle règle de nos documents testeriez-vous ?

Lire un test PHPUnit

Les prochains exemples illustrent les principes.
Le test unitaire utilise une classe fictive.
Les deux autres reprennent notre API Laravel.

Les extraits sont placés dans une classe de test

Un test qui ne vérifie rien

```
public function test_liste(): void
{
    $this->getJson('/api/documents');
    $this->assertTrue(true);
}
```

Exemple volontairement mauvais

Pourquoi est-il trompeur ?

getJson envoie une requête à l'API.
Son résultat n'est jamais vérifié.
assertTrue(true) réussit toujours.
Le test peut donc être vert malgré une erreur.

Un bon test vérifie un résultat lié à une règle

Une petite règle fictive

```
final class TitreDocument
{
    public static function estValide(string $titre): bool
    {
        return trim($titre) !== '';
    }
}
```

Classe pédagogique fictive, absente du projet

Vérifier un titre vide

```
public function test_titre_vide_est_refuse(): void
{
    $resultat = TitreDocument::estValide('   ');
    $this->assertFalse($resultat);
}
```

Extrait d’un test PHPUnit de la classe fictive

Explication ligne par ligne

test_titre_vide_est_refuse : nom du scénario.
estValide : appel de la règle avec des espaces.
$resultat reçoit la réponse de la règle.
assertFalse vérifie que la réponse est fausse.

Pas de requête HTTP ni de base de données

La liste exige une connexion

```
public function test_liste_sans_connexion(): void
{
    $response = $this->getJson('/api/documents');
    $response->assertUnauthorized();
}
```

Extrait d’un test Laravel : réponse attendue 401

Explication ligne par ligne

getJson envoie un GET vers la route de liste.
$response conserve la réponse de Laravel.
assertUnauthorized vérifie le statut HTTP 401.
Sans connexion, la liste ne doit pas être accessible.

Une requête et une réponse de l’application

Créer et retrouver un document

```
public function test_creation_enregistree(): void
{
    $user = User::factory()->create();
    $this->actingAs($user, 'sanctum')
        ->postJson('/api/documents', [
            'title' => 'Contrat', 'category' => 'Formation',
        ])->assertCreated();
    $this->assertDatabaseHas('documents', [
        'title' => 'Contrat', 'user_id' => $user->id,
    ]);
}
```

Extrait Laravel : API et base de test

Préparer et agir

User::factory()->create() crée un compte de test.
actingAs identifie ce compte avec Sanctum.
postJson envoie le titre et la catégorie.
assertCreated vérifie la réponse HTTP 201.

Plusieurs éléments Laravel travaillent ensemble

Vérifier la base de données

assertDatabaseHas cherche une ligne correspondante.
Le titre doit être « Contrat ».
user_id doit être l’identifiant du compte créé.
La réponse 201 seule ne vérifie pas ces données.

Le test vérifie la réponse et l’enregistrement

COMPARAISON

Trois portées de test

Unitaire : une règle isolée, sans base.
Fonctionnel : un comportement de l'API.
Intégration : plusieurs composants collaborent.
Les tests fonctionnels peuvent aussi être intégrés.

Les catégories peuvent se recouper

Ce qui rend un test utile

Un nom décrit le comportement attendu.
Les données sont maîtrisées et reproductibles.
Une assertion vérifie réellement la règle.
Le test échoue si ce comportement se casse.

Un test vert aide seulement sur ce qu’il vérifie

Préparer les classes de test

Unitaire : classe PHPUnit et règle fictive.
Laravel : Tests\TestCase pour les requêtes.
Base : RefreshDatabase et configuration isolée.
Les imports de User et des outils sont nécessaires.

Ces exemples servent à lire le code en classe